APPENDIX TO "RABIT-KV: A PHYSICALLY PACKED, TARGET-BIT-AWARE KV CACHE FOR LLM SERVING"

Ryan Zhang

This appendix gives the full tables behind the compact results of the main paper and the details of its accounting, validation, and diagnostics. Section and table references to "the main paper" are to the submitted manuscript.

A FULL CONTEXT-SCALING RESULTS

Table 1 gives every cell of the context-scaling experiment (single request, 32 greedily decoded output tokens, 15 samples per cell, one session, Llama-3.1-8B). Allocator capacity is the same at every prompt length (393,024 tokens for BF16 and 2,074,592 for RABIT-KV).

	TPOT (ms)			TTFT (ms)	
Prompt	BF16	RABIT-KV	Δ	BF16	RABIT-KV
512	15.33	21.26	+38.7%	32.1	95.6
2,048	15.56	21.00	+35.0%	59.8	119.9
4,096	15.60	21.03	+34.8%	144.6	171.3
8,192	15.72	20.66	+31.4%	409.5	305.2
16,384	15.43	21.15	+37.1%	1,280.8	687.8
32,736	15.56	29.38	+88.9%	4,435.0	114,376.8

Table 1. Context scaling (medians). The 32,736-token prompt is processed as a 16,384-token first chunk and a 16,352-token later chunk.

Two remarks. First, at 8,192 and 16,384 tokens the RABIT-KV TTFT is lower than that of BF16. Both methods compute the first prefill chunk densely, and we have not analyzed the difference; it is not a general prefill advantage, since TTFT is higher at shorter prompts and about 26 times higher at the 32,736-token point. One untested hypothesis is that the two paths use different dense attention kernels for the initial prefill, as they did in the separate serving-perplexity runs (Section E); we did not check this for these runs and do not attribute the difference to it. Second, 32,736 tokens is the nominal model limit of our configuration (32,768 including the output tokens), not a demonstrated maximum context. Absolute latencies in this table come from a different session than the matched four-method comparison of the main paper and must not be compared with it.

Preprint. Code, data, and evidence: https://github.com/ryanz090504-star/RABIT-KV.

Dispersion. Each session is summarized on its own; samples are never pooled across sessions. In this session every cell has 15 samples, and the full range (maximum minus minimum) is at most 6.2% of the cell median for TPOT and at most 8.4% for TTFT. In the matched four-method session of the main paper (two legs of 30 samples per method), the two leg medians of TPOT are 24.04 and 24.34 ms for BF16, 25.62 and 25.88 for FP8, 27.42 and 28.16 for TurboQuant, and 29.75 and 30.45 for RABIT-KV; within each leg, the 90th percentile is at most 1.9% above the median.

B FULL CONCURRENCY RESULTS

Tables 2 and 3 give all points of the concurrency experiment: a closed loop that keeps at most C requests in flight, 256 measured requests per point after an unmeasured conditioning batch, 32 output tokens per request, three trials per point (medians reported), a fresh engine process per point. No preemption was observed.

	Throughput (req/s)			TPOT (ms)	
C	BF16	RABIT-KV	ratio	BF16	RABIT-KV
1	1.458	1.022	0.70	20	26
4	4.592	1.612	0.35	21	64
8	7.218	1.844	0.26	21	111
16	10.042	2.015	0.20	35	219
32	12.911	2.083	0.16	52	432
64	14.361	1.776	0.12	108	985

Table 2. Concurrency at 2048-token prompts. Both methods sustain every tested concurrency. RABIT-KV enters the later-chunk prefill path for $C \geq 16$.

The later-chunk path is entered whenever the product of concurrency and prompt length exceeds the per-step token budget of 16,384 tokens, because the budget is then shared among requests and a prompt is split across scheduler steps. These runs do not separate the time of later-chunk prefills from that of the decoding steps scheduled with them. One untested hypothesis for the steep TPOT increase of RABIT-KV beyond C=16 at 8,192 tokens is that its serialized later-chunk prefills delay co-scheduled decoding; we did not measure this, and the TPOT values above include any such delay.

	Throughput (req/s)			TPOT (ms)	
C	BF16	RABIT-KV	ratio	BF16	RABIT-KV
1	0.937	0.828	0.88	21	28
4	1.801	1.259	0.70	46	82
8	2.101	1.385	0.66	96	162
16	2.282	1.424	0.62	188	312
32	2.346	0.870	0.37	372	1,033
64	2.366[a]	0.323	—	571	6,487

Table 3. Concurrency at 8192-token prompts. RABIT-KV enters the later-chunk prefill path for $C \geq 4$. [a]BF16 does not reach the offered concurrency: at most 47 requests are in flight in each of the three trials, the allocator bound for this prompt length, whereas RABIT-KV has 64 in flight. The two C=64 entries are therefore measured under unequal load, and we give no ratio.

Dispersion. Each entry is the median of three trial-level values, each the median over that trial's 256 requests; per-request samples are not pooled across trials. Over all points, the range across the three trials (maximum minus minimum) has a median of 7% of the reported throughput and 9% of the reported TPOT, but it reaches 52% for RABIT-KV throughput at 8,192 tokens and C=64 and 76% for BF16 TPOT at 8,192 tokens and C=1; BF16 throughput at that point ranges from 0.710 to 1.057 req/s. Throughput ratios close to one are therefore not resolved by three trials.

C PHYSICAL STORAGE ACCOUNTING

Page bytes. For a model with H KV heads of dimension d (a multiple of 32, with Hd a multiple of 64 so that the metadata groups below divide each array exactly; the serving engine further requires d to be a multiple of 64), one 32-token page of one layer stores:

- key payload: $32 \cdot H \cdot d$ elements at 3 bits, $12Hd$ bytes;
- value payload: $32 \cdot H \cdot d$ elements at 2 bits, $8Hd$ bytes;
- four metadata arrays (key minimum, key scale, value minimum, value scale), each with Hd primary values: one per (head, channel) for keys, and one per (token, head, 32-channel group) for values. Each array is stored as Hd 8-bit codes plus one BF16 minimum and one BF16 scale per 64 values, $Hd + 4 \cdot Hd/64$ bytes.

The page therefore occupies $12Hd + 8Hd + 4(Hd + Hd/16) = 24.25\,Hd$ bytes. Table 4 instantiates this for the two models.

Codec details. The rules below are those of the evaluator and the independent reference, which the serving engine matches bit for bit (Section D).

- *Primary parameters.* Per group, the minimum and maximum are taken in FP32 over its 32 elements. The scale is $(\max - \min)/7$ for keys and $(\max - \min)/3$ for values, replaced by 1 when its magnitude is below 10^{-8} (a zero range). Codes are $\mathrm{round}((x - \min)/s)$, clamped to $[0, 7]$ or $[0, 3]$, rounding to nearest with ties to even; they use the FP32 primary minimum and scale.
- *META8g64.* Each primary array is flattened in a fixed order, (head, 32-token group, channel) for keys and (token, head, 32-channel group) for values, and cut into groups of 64 consecutive values. A group's scale is $(\max - \min)/255$, replaced by 1 below 10^{-12}; its codes are $\mathrm{round}((p - \min)/s)$, clamped to $[0, 255]$ and stored as unsigned bytes, and its minimum and scale are stored in BF16. Because Hd is a multiple of 64, every metadata group of a closed page lies within that page. An array whose length is not a multiple of 64, which occurs only in the open group, is padded with copies of its last value.
- *Dequantization.* A key or value is $c\,\hat{s} + \hat{m}$ in FP32, where $\hat{s}$ and $\hat{m}$ are the scale and minimum decoded from META8g64. Codes thus come from the primary parameters and values from the decoded ones.
- *Open group.* The serving engine keeps the open group's keys exactly, in BF16. At each read it quantizes them as one 32-token group zero-padded to 32 tokens, with META8g64 applied to the resulting parameters, and decodes only the real tokens; the padding affects only the minimum and maximum. The open group's values are stored as 2-bit codes with their FP32 primary minimum and scale, and META8g64 is applied to these at each read and once more when the page closes. A value's codes and primary parameters are therefore fixed when its token leaves the residual window, but its decoded value is not. In (token, head, 32-channel group) order, a 64-value metadata group holds the primaries of $64/(Hd/32)$ consecutive tokens: two on Llama-3.1-8B and four on Qwen2.5-7B. Until all of them have entered the open group, the group is padded with its last value, and each arriving token can change the group's minimum, scale, and codes, and with them the decoded values of the earlier tokens in the group. Once the group is full its encoding no longer changes, and it equals the encoding written to the page, because the open group starts at a page boundary and the metadata groups are aligned to it. We did not measure the size of these changes. When the group reaches 32 tokens, its keys are quantized without padding and the page is written and never changed. The evaluator recomputes the open group and the residual window from the raw BF16 tokens at every length, which gives the same state.

We give no design rationale for the zero padding. It is part of the reference definition that the evaluator and the serving engine reproduce; Section H measures its effect.

Later prefill chunks. For a later prefill chunk, the engine first encodes, from the exact keys and values of the open group and of the chunk, every page that closes within the chunk and writes it into its scheduler-assigned block. It then processes the chunk's queries in order. Before each query, the visible state is set to exactly the state after appending that query's token: the number of closed pages, the open group, and the residual window. A query therefore never sees a page that closes later in the chunk, and the lifecycle advances token by token as in decoding. In the default implementation, each query then runs the decode-path attention over that state. The context-scaling and concurrency runs used an optional implementation in which consecutive queries with the same visible closed pages (at most 32, ending at each page closure) share one decode of each closed page, while the state update and the open-group and residual attention stay per query. The serving-perplexity runs used the default implementation.

	Llama-3.1-8B	Qwen2.5-7B
Layers, H, d	32, 8, 128	28, 4, 128
Key payload (B)	12,288	6,144
Value payload (B)	8,192	4,096
Metadata, 4 arrays (B)	$4 \times 1{,}088$	4×544
Page, one layer (B)	24,832	12,416
Per token, one layer (B)	776	388
Per token, all layers (B)	24,832	10,864
BF16 per token, all layers (B)	131,072	57,344

Table 4. Page accounting for the two model geometries.

Allocator-observed capacity. Table 5 lists what the running engine reports. The block and token counts are measured. The pool size in bytes is the block count times the bytes of one block in each method's layout (32 tokens over all layers), so it is exact but derived. The engine logs its available KV budget only to 0.01 GiB, and the budget in bytes was not recorded; every pool fits within the logged value. The measured capacity ratio over BF16 is 5.2785 for both models.

What the capacity does not include. The residual window and the open group of each active sequence, at most 35 tokens per sequence, are kept outside the block pool and are not part of these numbers. The live memory of a running workload was not measured; only the size of the pool was.

Decomposition of pool bytes. Table 6 splits the pool bytes per token of each tested method into quantized codes, quantization metadata, and layers kept in BF16, from each method's block layout in our engine snapshot. The tested TurboQuant preset stores per head and token 3-bit key codes with a 16-bit norm and 4-bit value codes with a 16-bit scale and zero point (118 bytes over $H{=}8$ heads, 944 bytes per layer), in 28 layers. Its four BF16 boundary layers account for 38% of its pool bytes, and metadata for 3%. RABIT-KV keeps no layer in BF16, but its metadata is 18% of its pool bytes. For comparison only: a TurboQuant configuration that also quantized the four boundary layers would take $32 \times 944 = 30{,}208$ bytes per token, $4.34\times$ BF16, and RABIT-KV's pool would then hold $1.22\times$ its tokens rather than $1.72\times$. This is arithmetic on the layout. We did not configure or run such a setting, and its quality and performance are unknown.

Method	Blocks	Tokens	Pool bytes	Log (GiB)
Llama-3.1-8B				
BF16	12,282	393,024	51,514,441,728	47.98
FP8	24,549	785,568	51,482,984,448	47.95
TurboQuant[a]	37,600	1,203,200	51,516,211,200	47.98
RABIT-KV	64,831	2,074,592	51,516,268,544	47.98
Qwen2.5-7B				
BF16	28,208	902,656	51,761,905,664	48.21
RABIT-KV	148,895	4,764,640	51,763,048,960	48.21

Table 5. Allocator-observed capacity. Pool bytes: blocks times block bytes of the layout (exact, derived). Log: available KV memory as printed by the engine, rounded to 0.01 GiB. [a]Tested configuration.

Method	Codes	Metadata	BF16 layers	Total
BF16	131,072	0	0	131,072
FP8	65,536	0	0	65,536
TurboQuant[a]	25,088	1,344	16,384	42,816
RABIT-KV	20,480	4,352	0	24,832

Table 6. Pool bytes per token over all 32 layers of Llama-3.1-8B, by component. Derived from the block layouts; the totals equal the paged-pool bytes of the main paper's capacity table. [a]Tested configuration.

From the target label to closed-page bits. Per key and value element, a closed RABIT-KV page holds 1.5 bits of key payload, 1.0 bit of value payload, and 0.53 bits of META8g64 metadata, 3.03 bits in total. The following are counterfactual layouts, computed with the same formula and neither built nor evaluated for quality: storing the group parameters in BF16 instead of META8g64 would give 3.5 bits and a pool ratio of $4.57\times$ over BF16; the 2.5 payload bits alone would give $6.40\times$; 64-element groups with META8g64 would give $5.79\times$. The last is not a metadata change alone: key groups would span 64 tokens, so a page and an allocator block would hold 64 tokens and the zero-padded open group up to 63; value groups would span

64 channels; and the per-sequence state below would grow from 96 to 168 KiB per layer. These counterfactuals show how much of the gap between the label and the capacity is metadata; they do not show that any of these layouts would retain quality.

Per-sequence state outside the pool. The current implementation keeps the residual window and the open group per request and layer. Once a sequence decodes, they occupy fixed buffers: four BF16 residual tokens (16 KiB for keys and values), 32 BF16 open-group keys (64 KiB, kept exact so that the group can be re-quantized), and 32 tokens of packed 2-bit values with FP32 primary minima and scales (16 KiB); before that, during prefill, the same state is held in tensors of at most this size. That is 96 KiB per layer and 3 MiB per Llama-3.1-8B sequence, about 24 BF16 tokens' worth, in requested tensor bytes, excluding allocator rounding and workspaces shared by all sequences. The scheduler still reserves pool blocks for every token of a sequence, including those held in this state, so the state adds to the pool rather than replacing part of it. For one sequence, the ratio of its BF16 pool bytes to its RABIT-KV pool bytes plus this state is $2.65\times$ at 128 tokens, $4.23\times$ at 512, $4.70\times$ at 1,024, $5.12\times$ at 4,096, $5.20\times$ at 8,192, and $5.26\times$ at 32,768 tokens; 64 decoding sequences hold 192 MiB. These figures are derived from the source code and were not measured, and this memory lies outside the KV budget by which the capacity of the main paper is sized.

A token-selection index under the same accounting. RaBitQCache (main paper, Related Work) keeps every token's KV state in full precision and adds a key index. In its released vLLM integration, the paged pool keeps the full-attention page size, so pool capacity equals BF16's, and the index is allocated per request beside the pool. Per token slot and KV head of an indexed layer it holds 128 one-byte unpacked sign codes, 16 bytes of packed sign bits, one FP32 sum, and three 16-bit scalars: 154 bytes, against 512 bytes of BF16 keys and values (30%); slots are reserved at prefill for the prompt plus the larger of 512 tokens and ten times its length. A layout with only the packed bits and the three scalars would take 22 bytes per token, 4.3%. Such an index addresses the bytes read per decoding step, not the bytes stored, and could in principle be built over a low-bit cache; we have not evaluated that combination.

D Correctness and Reference Conformance

Two implementations are tied to one independently written reference implementation of the representation: the serving implementation, through its test suite, and the quality evaluator, through bit-exact conformance checks. Table 7 summarizes the checks.

Check	Result
Serving test suite (layout, page codec, online state, metadata, grouped-query attention, causal chunked prefill), run before the serving experiments	105 tests pass
Evaluator vs. reference, CPU, synthetic states	276 of 276 cases bit-exact, including 56 of 56 token-by-token aging cases
Evaluator vs. reference, GPU, Llama-3.1-8B keys and values (1,024-token prefill)	32 of 32 layers, every field bit-exact
Evaluator vs. reference, GPU, Qwen2.5-7B keys and values	28 of 28 layers, every field bit-exact
Evaluator vs. reference, GPU, Llama-3.1-8B at 16,383 tokens (511 closed pages, 27 open tokens, 4 residual)	32 of 32 layers, 0 differing elements
Online aging, GPU, synthetic geometries	28 of 28 cases per geometry in each run

Table 7. Conformance with the independent reference. All comparisons are made on one device.

CPU and GPU are not bit-identical. The state computed on the CPU and on the GPU from the same keys and values differs slightly. The earliest difference is in the single-precision group scale, which changes a small fraction of codes by one level (for Llama-3.1-8B at 1,024 tokens, 0.074% of key codes and 0.020% of value codes). For this reason conformance is established separately on each device against the reference running on that device, and no cross-device tolerance is defined or used.

Scope. These checks establish that the evaluator computes the intended stored state and that the serving implementation agrees with the reference on its tested cases. They are not a formal verification, and serving outputs were not compared with evaluator outputs end to end.

E Quality Details

Continuation perplexity. Each model is scored on 32 consecutive, non-overlapping WikiText-2 test windows of 1,024 context tokens and 128 scored tokens (4,096 scored tokens per arm). Perplexity is the exponential of the total negative log-likelihood divided by the number of scored tokens; the relative difference is computed from the mean per-window difference in negative log-likelihood, with a paired percentile bootstrap over windows. Table 8 gives the distribution over windows.

For Llama-3.1-8B the loss is spread over the windows and not driven by a few: the three windows with the largest difference account for 28% of the summed difference, and the relative difference without them is $+1.24\%$.

	Llama-3.1-8B	Qwen2.5-7B
BF16 perplexity	7.515	6.787
RABIT-KV perplexity	7.633	112.39
Relative difference	$+1.56\%$	$+1555.9\%$
Paired 95% CI	$[+1.12, +2.05]\%$	$[+1320, +1828]\%$
ΔNLL, mean	0.0155	2.807
ΔNLL, median	0.0136	2.795
ΔNLL, minimum	-0.0063	1.931
ΔNLL, maximum	0.0590	3.724
Windows worse / better	30 / 2	32 / 0

Table 8. Continuation perplexity by model. ΔNLL is the per-window mean negative log-likelihood of RABIT-KV minus that of BF16.

Long-context tasks (Llama-3.1-8B). Prompts are truncated to at most 16,384 tokens (the first and last 8,192 tokens of longer inputs). Generation is greedy with at most 16 new tokens for the needle task and 32 for the LongBench tasks. Whether a generated token reaches a closed page depends on the prompt length modulo 32 and on how many tokens are generated before the end-of-sequence token, not on a fixed number of steps: a page closes when the open group reaches 32 tokens, which can include tokens of the prompt. In the registered run, a page closed during decoding in all 57 needle cases, 23 of 200 Passage Retrieval examples, and 17 of 100 HotpotQA examples, but a generated token reached a closed page only in 4 HotpotQA examples (42 tokens in total, at most 24 in one example). These tasks therefore mainly test attention over the converted prompt state, not the aging of generated tokens across many pages.

- *Needle in a haystack.* A secret code is inserted into WikiText-2 filler text at 19 depths (5% to 95%) for contexts of 4,096, 8,192, and 16,384 tokens, 57 cases in total; a case is correct if the exact code is returned. BF16 and RABIT-KV are correct in 19 of 19 cases at each length. The generated text differs between the arms in 23 cases without changing any score. The cases share one needle and one filler construction and are not independent draws, so no confidence interval is given.
- *Passage Retrieval* (200 examples). Both arms score 100.0; no individual score changes (the predicted text differs in 79 examples).
- *HotpotQA* (100 examples of the long-input subset). Table 9 gives both scorers. The primary scorer does not strip English articles before computing token F1; the secondary scorer is the standard LongBench answer F1. Both were fixed before the run and are computed from the same predictions. No example is excluded.

The bootstrap is conditional on the greedy generations of this run and does not model run-to-run variation of GPU arithmetic. Long-context tasks were not run for Qwen2.5-7B.

	Primary	Secondary
BF16 F1	59.30	59.85
RABIT-KV F1	58.27	58.71
Difference (points)	-1.03	-1.14
Paired 95% CI	$[-4.24, +2.20]$	$[-4.30, +2.03]$
Identical / worse / better	92 / 5 / 3	93 / 5 / 2
Median paired difference	0.0	0.0

Table 9. HotpotQA under both scorers. The interval spans zero in both cases, which does not establish equivalence at this sample size.

Perplexity in the serving engine (Llama-3.1-8B). This measurement is separate from Table 8 and was made after the main quality runs. It scores continuation perplexity from the serving engine itself, on the 32 windows of Table 8: 1,024 context tokens and 128 scored tokens per window, with no beginning-of-sequence token. Each continuation token is fed through a real decode step and scored from the engine's original logits before the sampled token is replaced by the reference token. Four cells run in fresh engine processes in a fixed order: BF16 and RABIT-KV pages, each with the context prefilled in one 1,024-token chunk and with a per-step budget of 512 tokens, which splits it into two chunks so that the second attends to an already packed prefix. Engine settings are those of the serving experiments, with one request at a time. An independent BF16 reference in Hugging Face Transformers scores the same tokens. The model files were those of the revision used for the quality evaluation, hashed before loading.

Protocol. Inputs, cells, windows, endpoints, and the analysis were fixed in a private, self-attested freeze record before the run. Windows 5–32 are the primary set, with two co-primary endpoints (one chunk and two chunks, each RABIT-KV against BF16 under the same schedule) and 97.5% paired window-bootstrap intervals (100,000 resamples). Windows 1–4 had been scored in an earlier four-window pilot; they and all 32 windows are secondary, with 95% intervals. An earlier attempt under the same freeze was cancelled by a client disconnection after the BF16 one-chunk cell had scored all 32 windows and the RABIT-KV one-chunk cell windows 1–23 and part of window 24. Those outcomes were seen before the reported run, which repeated all four cells; nothing from the cancelled attempt is pooled. The windows come from the pool used throughout the paper and are not held out.

Validity checks. All passed: token identities and scoring positions; agreement of the scores with the engine's own log-probabilities within 2×10^{-5}; the actual prefill sched-

Windows	5–32	1–4	1–32
BF16	7.493	7.718	7.521
RABIT-KV, 1 chunk	7.600	7.843	7.630
RABIT-KV, 2 chunks	7.640	7.940	7.677
1 chunk, change	$+1.42\%$	$+1.61\%$	$+1.44\%$
interval	$[+0.86, +2.00]$	$[+0.47, +2.90]$	$[+0.99, +1.91]$
2 chunks, change	$+1.96\%$	$+2.87\%$	$+2.07\%$
interval	$[+1.07, +2.93]$	$[+1.55, +3.91]$	$[+1.36, +2.83]$

Table 10. Continuation perplexity scored in the serving engine, Llama-3.1-8B. Windows 5–32 are primary. BF16 perplexity is identical under both schedules. Intervals are paired window bootstraps: 97.5% for the primary windows, 95% otherwise. On the primary windows, resampling blocks of four contiguous windows gives $[+0.91, +1.88]$ and $[+1.37, +2.59]$.

ules; for RABIT-KV, packed pages of 24,832 bytes whose writes are read back byte for byte, reads only of pages written earlier for the same request and layer, the packed attention kernel on every decode step, and no fallback path; packed attention within 0.025 of an independent dense reference at checkpoints; and BF16 serving perplexity within 0.5% of the independent reference ($+0.17\%$). The BF16 scores are identical under both schedules at all 4,096 positions. Table 10 gives the results.

Interpretation. These are end-to-end differences of the serving engine, not a quantization-only effect. The BF16 and RABIT-KV cells use different dense attention kernels for the initial prefill; on identical inputs their layer-0 outputs differ by at most 9.8×10^{-4}, and the difference propagates. A post-hoc control outside the protocol ran BF16 with the prefill kernel of the RABIT-KV path on all 32 windows; the differences became $+1.51\%$ and $+2.18\%$, so they are not explained by that kernel, but the decode kernels still differ. The difference between the two schedules, $+0.53\%$ on the primary windows (95% CI -0.08% to $+1.19\%$), is not established. The serving BF16 perplexity (7.521 on all 32 windows) differs from that of the evaluator (7.515) because the implementations differ. The instrumentation synchronizes and reads back device memory, so these runs carry no latency, throughput, or capacity information. They cover Llama-3.1-8B and continuation perplexity only: no generation task, no Qwen2.5-7B, no prompt beyond the per-step token budget, and no concurrent requests.

F PERFORMANCE DIAGNOSTIC

This section describes a profiling diagnostic that we use only to explain the scaling results. It contains no performance result; every latency and throughput number in the paper comes from unprofiled runs.

Setup. Three workloads were profiled on Llama-3.1-8B with the unmodified serving implementation, each with an unprofiled reference run and a BF16 run of the same shape: (1) 2048-token prompts at $C{=}8$, where the later-chunk prefill path is not entered; (2) 8192-token prompts at $C{=}32$; (3) one 32,736-token request. Request counts were reduced relative to the main experiments. The profiler attached to the engine from outside and timed scheduler steps, attention layers, the execution paths of Section 4 of the main paper, and every kernel launch, in two separate domains (host time and device time spans).

The profiler perturbed execution. Table 11 shows that the profiled RABIT-KV runs were 3.7 to 8.8 times slower than the unprofiled ones. After subtracting the profiler's own recorded bookkeeping, the remaining time is still 1.35 to 1.63 times the unprofiled time. We therefore report no component percentages: they are not performance measurements. The unprofiled runs, in contrast, agree with the main experiments (1.848 against 1.844 req/s, 0.900 against 0.870 req/s, and a TTFT of 118.3 s against 114.4 s), which confirms that the diagnostic exercised the same workloads.

Workload	Unprofiled (s)	Profiled (s)	Ratio
2048 tokens, $C{=}8$	51.9	194.2	$3.74\times$
8192 tokens, $C{=}32$	106.7	670.8	$6.29\times$
32,736 tokens, one request	119.3	1,051.7	$8.82\times$

Table 11. Wall time of the measured phase of the RABIT-KV runs with and without the profiler.

What the diagnostic indicates. The qualitative picture is consistent across the three workloads, the source code, and one cross-check with a framework profiler.

- Work is serialized per request and per token. In decode, each (layer, request, token) triple performs about 10.8 kernel launches: appending and aging, requantizing the open key group and re-encoding its metadata, the open-tail attention, the closed-page attention, and a reduction. Inside a later prefill chunk, each (layer, token) pair performs about 6.9 launches. These counts are exact; time shares are not.
- The 32,736-token request executes 523,264 such (layer, token) iterations in its later chunk.

- At 32,736 tokens, the closed-page attention of the later chunk additionally accounts for device work on the order of tens of seconds, so removing the host-side serialization alone would not remove the long-context cost.

We do not claim that these costs are intrinsic to the representation. Removing them requires batching across requests and tokens, which the current implementation does not do.

G Status of Earlier Quality Results

During development, quality was measured with an earlier logical evaluator. A later audit against the reference implementation found that it did not reproduce the representation exactly, in two respects. First, it flattened the value metadata in (head, token, group) order before the second-level quantization, whereas the representation uses (token, head, group) order, so the 64-value metadata groups contained different values. Second, it quantized only the prompt, once, and left the last prompt token and all generated tokens in full precision, whereas in the representation every token ages through the residual window, the open group, and a closed page.

All quality numbers in the main paper come from the corrected evaluator, which is validated as described in Section D. Results obtained with the earlier evaluator are *legacy logical-evaluator results*. They are not pooled with, or compared against, the results of the main paper, and we draw no conclusion from them. In particular, the component studies carried out during development (key and value bit-width, group size, residual length, and metadata precision) exist only under the earlier evaluator and were not repeated, except metadata precision (Section I); this is why the main paper does not claim that the individual design choices are optimal. We did not decompose how much each of the two differences contributes to the change in measured quality.

H Post-hoc Diagnostic of the Qwen2.5-7B Degradation

This section reports diagnostics designed after the quality results of the main paper were known. They lie outside the registered protocol, change no number in the main paper, and use the logical evaluator only; the serving engine was not modified and no serving output was scored. So that paired arms repeat bit for bit, attention was fixed to the FlashAttention SDPA backend, whereas the registered runs select the backend automatically. The perplexities below are therefore not interchangeable with Table 8. Confidence intervals are paired percentile bootstraps over windows with 20,000 resamples and, except in the replication below, are not adjusted for the number of contrasts.

Open-group padding. The frozen policy quantizes the open key group as a 32-token group padded with zeros, so the range of each channel includes zero even when the real keys do not. A post-hoc variant replaces the padding with copies of the last real key, so that the range comes from real keys only; closed pages, values, metadata, and the residual window are unchanged. Over the 32 windows, Qwen2.5-7B perplexity is 113.93 under the frozen policy, 7.84 with padding excluded, 6.86 with exact keys (frozen values and metadata), and 6.79 for BF16. Padding exclusion improves all 32 windows and removes 94.9% $[93.8, 96.0]$ of the excess negative log-likelihood, but remains $+15.5\%$ $[+12.1, +19.1]$ above BF16. For Llama-3.1-8B it changes perplexity by -0.26% $[-0.56, +0.04]$ relative to the frozen policy.

Location of the remaining key error. A second diagnostic, registered before any of its arms was scored, uses eight evenly spaced windows $(1, 5, \ldots, 29)$ of the same 32; none is held out. Starting from padding exclusion, it makes exact either all keys, only the keys of the open group, or only the keys of the closed pages; values, metadata, and the residual window are frozen in every arm (Table 12).

For Qwen2.5-7B, exact closed-page keys alone remove a fraction 1.016 $[0.913, 1.073]$ of the gap between padding exclusion and exact keys, and are better in seven of the eight windows; exact open-group keys remove -0.017 $[-0.097, 0.074]$. These fractions are descriptive, not an additive decomposition, because each intervention changes all later activations. For Llama-3.1-8B the gap itself is not distinguishable from zero, so the fractions are not interpretable, and no arm differs detectably from padding exclusion. With exact keys, perplexity remains above BF16 for both models ($+1.21\%$ $[+0.21, +2.04]$ and $+1.28\%$ $[+0.49, +2.05]$); this design does not localize that remainder. All arms share pinned inputs; a pass-through cache equals BF16 bit for bit, an independently written reference reproduces every key arm bit for bit over all decode steps of one window, and cache checks pass at nine decode steps in every layer. The BF16, padding-excluded, and exact-key losses repeat bit for bit across two cloud H100 environments with different driver versions.

Replication on the validation split. Both Qwen2.5-7B findings were then tested on the WikiText-2 validation split. The windows, arms, controls, analysis, and criteria were fixed before any validation window was scored and frozen in a private, timestamped record of file hashes; the timestamp is self-attested, and the record is not a public preregistration. The 32 validation windows are built like the

Keys	Qwen2.5-7B		Llama-3.1-8B	
	PPL	vs. padding excl.	PPL	vs. padding excl.
BF16 cache	8.003		7.399	
Padding excluded	8.956		7.479	
Exact	8.100	-9.56% $[-14.1, -4.8]$	7.494	$+0.20\%$ $[-0.11, +0.50]$
Exact open group	8.971	$+0.17\%$ $[-0.58, +1.02]$	7.498	$+0.26\%$ $[-0.25, +0.79]$
Exact closed pages	8.087	-9.70% $[-14.6, -4.6]$	7.507	$+0.38\%$ $[-0.30, +1.11]$

Table 12. Post-hoc key-region diagnostic on eight windows, logical evaluator, fixed FlashAttention backend. Values, metadata, and the residual window are frozen in all quantized arms. Not comparable to Table 8.

test windows and share no 32-token sequence with the test split, but earlier use of the validation split cannot be excluded, so this is a replication on disjoint windows, not held-out or confirmatory validation. All six arms above are scored on all 32 windows. The window-1 controls must match bit for bit, and test window 1 must repeat the earlier BF16, padding-excluded, and exact-key losses bit for bit. Decisions use two-sided 97.5% paired-bootstrap intervals (Bonferroni over the two findings; seed and 20,000 resamples fixed in advance). The thresholds were set below the post-hoc estimates, so the test asks whether each finding recurs, not whether its magnitude does. The single run, on one H100, passed every control and anchor, and the criteria for both findings were met (Table 13). Padding exclusion and exact closed-page keys each improve all 32 windows. For Llama-3.1-8B, padding exclusion changes perplexity by -0.12% (95% interval $[-0.39, +0.15]$), within the prespecified secondary bound of $+0.5\%$. An independent recomputation from the stored per-token losses reproduces every estimate and decision. The in-run audit and its rerun on a second operating system reach identical decisions; their derived percentages differ by at most two units in the last place.

Scope. These results locate the excess loss within the stored key state of the logical evaluator. The exact-key arms are diagnostic controls that store keys in BF16, not deployable policies. Padding exclusion is not implemented in the serving engine, so its serving quality, capacity, and latency are unmeasured. Neither the diagnostic windows nor the replication split is held out, and the replication uses the same corpus, models, and evaluator, so it does not show that the findings hold on other data. The attention backend differs from the registered runs, and the diagnostic does not explain why the keys of Qwen2.5-7B are more sensitive than those of Llama-3.1-8B. The key projection of Qwen2.5-7B has a bias term and that of Llama-3.1-8B does not; whether this shifts key channels in a way that explains the difference is an untested hypothesis.

Finding	Pre-specified quantity	Estimate	97.5% interval	Criterion
Padding exclusion	Padding excluded vs. frozen policy, PPL	-91.4%	$[-92.9, -89.7]$	upper < 0
	Fraction of the frozen policy's excess NLL removed	0.941	$[0.924, 0.955]$	lower ≥ 0.80
Closed-page localization	Exact keys vs. padding excluded, PPL	-13.2%	$[-17.0, -9.6]$	upper < 0
	Exact closed pages vs. padding excluded, PPL	-12.8%	$[-16.7, -9.1]$	upper < 0
	Fraction of that gap removed by exact closed pages	0.963	$[0.901, 1.001]$	lower ≥ 0.75
	Fraction removed by exact open group	0.031	$[-0.021, 0.093]$	upper ≤ 0.25

Table 13. Pre-specified replication on 32 WikiText-2 validation windows, Qwen2.5-7B, logical evaluator, fixed FlashAttention backend; all criteria met. Perplexity: BF16 5.987, frozen policy 81.53, padding excluded 6.989, exact keys 6.066, exact closed pages 6.098, exact open group 6.958. Different split from Tables 8 and 12, so the perplexities are not comparable.

I Component Ablation: Metadata Precision

This ablation asks what the second-level 8-bit quantization of the group parameters (META8g64) costs in quality. It was designed after the results above were known, but its arms, windows, analysis, and reporting rule were fixed in a private, self-attested freeze record before any new arm was scored, and every result is reported. It uses the logical evaluator with the FlashAttention backend fixed, as in the diagnostic above, on the 32 WikiText-2 windows of the main evaluation; these windows are not held out.

Arms. Both models, all 32 windows: BF16; the frozen policy; padding exclusion only (as above); the frozen policy with exact metadata, in which the 3-bit key and 2-bit value codes are dequantized with the exact FP32 group minima and scales instead of their META8g64 encoding; and padding exclusion with exact metadata. The padding-only arm is the control that keeps the Qwen2.5-7B padding failure from masking the metadata effect. Nothing else changes: bit-widths, groups, residual window, and code computation are those of the operating point.

Validity checks. All passed. On window 1, pass-through equals BF16 and an independently implemented reference replay of each quantized arm, with the same padding and metadata variant, equals that arm bit for bit, and the cache equals the reference at nine decode steps in every layer. The BF16, frozen, and padding-excluded arms repeat the per-token losses of the earlier diagnostic run bit for bit on all 32 windows of both models. An independent local audit rebuilt the token pools and recomputed the analysis from the per-token losses; it agrees with the run's own audit except in the last floating-point bit of 10 of 86 reported values. All ten are percentage changes, at most 4.5×10^{-13} percentage points apart; no NLL difference, share, count, or validity flag differs. The byte-equality check between the two audits, written before the results were read, therefore failed; we report it as failed and did not relax it. The ablation has no decision rule that these values could affect.

Results. Table 14 reports perplexities and paired changes with 95% paired-window bootstrap intervals (20,000 resamples). For Llama-3.1-8B, exact metadata changes perplexity by $+0.02\%$ $[-0.28, +0.30]$ under the frozen policy and by $+0.06\%$ $[-0.19, +0.29]$ with padding excluded: the ablation detects no effect of META8g64 on this model. The intervals exclude changes larger than about 0.3%; they do not show that the effect is zero. For Qwen2.5-7B, exact metadata lowers perplexity by 12.07% $[9.43, 14.79]$ once padding is excluded, better in 30 of 32 windows. In mean token negative log-likelihood (log-perplexity), it removes 0.89 $[0.84, 0.95]$ of the excess of the padding-excluded arm over BF16 (0.129 of 0.144 nats per token). This is a ratio of NLL differences, not of perplexity increases. The excess over BF16 falls from $+15.47\%$ $[+12.15, +19.09]$ to $+1.53\%$ $[+0.72, +2.27]$.

Interaction. The effect of exact metadata depends on whether padding is excluded. Under the frozen policy, exact metadata lowers Qwen2.5-7B perplexity by 26.38% $[21.44, 30.90]$, yet this is only 0.11 $[0.09, 0.13]$ of that policy's excess NLL over BF16, because the padding failure dominates. The share of 0.89 is therefore conditional on padding exclusion. It is not an independent contribution, and shares attributed to padding and to metadata do not add up to one.

Limits. The exact-metadata arms store FP32 group parameters. That layout would occupy 1,152 bytes per token per layer, a pool ratio of $3.56\times$ over BF16 by the arithmetic of Section C, against $5.28\times$ for the operating point; a BF16-metadata layout ($4.57\times$) was not evaluated. Neither padding exclusion nor exact metadata is implemented in the serving engine, so no capacity, latency, throughput, or serving quality follows from these results, and no policy is adopted: the operating point of the main paper is unchanged. Paired contrasts are not additive decompositions. The results cover two models and one corpus.

	Llama-3.1-8B	Qwen2.5-7B
Perplexity, 32 windows		
BF16	7.514	6.791
Frozen policy	7.639	113.93
Excl. padding	7.619	7.841
Exact meta.	7.641	83.877
Excl. padding + exact meta.	7.623	6.895
Paired change in perplexity, %; windows worse/better		
Exact meta. vs. frozen	+0.02; 17/15	−26.38; 1/31
95% CI	[−0.28, +0.30]	[−30.90, −21.44]
Both vs. excl. padding	+0.06; 17/15	−12.07; 2/30
95% CI	[−0.19, +0.29]	[−14.79, −9.43]
Excl. padding vs. frozen	−0.26; 11/21	−93.12; 0/32
95% CI	[−0.56, +0.04]	[−94.09, −92.01]
Frozen vs. BF16	+1.67; 28/4	+1578; 32/0
95% CI	[+1.18, +2.20]	[+1346, +1854]
Exact meta. vs. BF16	+1.69; 31/1	+1135; 32/0
95% CI	[+1.32, +2.08]	[+956, +1351]
Excl. padding vs. BF16	+1.41; 27/5	+15.47; 31/1
95% CI	[+0.93, +1.91]	[+12.15, +19.09]
Both vs. BF16	+1.46; 28/4	+1.53; 26/6
95% CI	[+1.01, +1.93]	[+0.72, +2.27]
Share of excess NLL over BF16 removed		
With padding excluded	−0.04	0.89
95% CI	[−0.26, +0.13]	[0.84, 0.95]
Under the frozen policy	−0.01	0.11
95% CI	[−0.23, +0.14]	[0.09, 0.13]

Table 14. Metadata ablation, logical evaluator, FlashAttention backend fixed. Not interchangeable with Table 8. Exact meta.: dequantized with the FP32 group parameters; excl. padding: padding excluded; both: excl. padding + exact meta. Shares (removed by exact meta.) are ratios of differences in mean token NLL, conditional on the padding variant; they are not additive.

J Reproducibility Details

Hardware. One NVIDIA H100 80GB GPU per run.

Serving configuration. A development snapshot of vLLM with our cache type added; eager mode; 32-token blocks; GPU memory fraction 0.82; maximum model length 32,768; per-step token budget 16,384; chunked prefill enabled; prefix caching disabled; BF16 model weights. Latency experiments use 32 greedily decoded output tokens per request (temperature 0, end-of-sequence ignored). Concurrency experiments set the maximum number of sequences to C and start a fresh engine process for every point.

Baselines. BF16: the default cache. FP8: the engine's per-tensor FP8 (E4M3) cache with default scales. TurboQuant: the upstream vLLM backend, unmodified in our snapshot, with the preset of 3-bit keys and 4-bit values (with norm correction); by the backend's default the first two and last two layers stay in BF16, leaving 28 quantized layers.

Quality evaluation. Python 3.11, PyTorch 2.11.0 with CUDA 13.0, Transformers 4.48.2, BF16 weights, batch size 1. Models: Llama-3.1-8B-Instruct and Qwen2.5-7B-Instruct at pinned revisions whose file manifests were checked before every run. Data: the WikiText-2 raw test set; LongBench Passage Retrieval (English) and the long-input HotpotQA subset at pinned dataset revisions, with file hashes checked before every run. Confidence intervals: paired percentile bootstrap with fixed seeds; the number of resamples is the one fixed in each experiment's frozen analysis. It is 10,000 for the registered quality runs of the main paper, with these exceptions: 100,000 for the serving-engine perplexity run (Section E), and 20,000 for the post-hoc Qwen2.5-7B diagnostics, their replication on the validation split (Section H), and the metadata ablation (Section I).

Protocol. Every quality run was specified before execution (inputs, example selection, scorers, seeds, validity checks, and interpretation rules) and was accepted or excluded by its validity checks; the checks on output quality use only BF16 outputs. One perplexity run was excluded because it failed a validity check; its results are not used. For the quality experiments, model revisions and file manifests, dataset file hashes, protocol settings, and bootstrap seeds are recorded in the experiment artifacts. For the serving experiments, the artifacts record the engine configuration and hashes of the serving source; a pinned model revision was recorded only for the Qwen2.5-7B serving runs. The Llama-3.1-8B capacity, latency, and concurrency runs fetched the model by a moving branch reference and did not record a revision. For the capacity, latency, context-scaling, FP8, and TurboQuant runs, the logged hashes of the configuration, tokenizer, and all weight files match the pinned snapshot (revision 359efdbb) used for the quality evaluation and for the serving-engine perplexity. The concurrency runs logged no file hashes; their only provenance is that they read the same model cache, whose files a later audit found identical to that snapshot.